# 공공입찰 제안요청서 기반 질의응답 시스템 최종 보고서

API 활용 방식과 모델 직접 실행 방식의 구현 및 비교

파트3 3팀 | 나상훈 · 김도영 · 김시현 · 김연주 · 박단비 · 유찬혁

## 요약

본 프로젝트는 공공입찰 컨설턴트가 제안요청서에서 필요한 답과 원문 근거를 함께 찾을 수 있도록 문서 기반 검색 증강 생성(RAG) 시스템을 구현했다. 제안요청서는 사업 개요, 예산, 기간, 참가 자격, 제출 조건 등이 긴 본문과 표에 흩어져 있다. 이에 원본에서 본문과 표를 추출하고, 질문과 관련된 내용을 검색한 뒤 해당 근거로 답변과 출처를 제공하는 흐름을 구성했다.

외부 모델 API를 활용하는 방식과 클라우드 서버에서 모델을 직접 실행하는 방식을 모두 구현했다. 두 방식 각각에서 문서 분할, 임베딩, 검색, 답변 모델과 근거 전달 설정을 비교하고 최종 구성을 정했다. API 활용 방식은 문서 구조를 반영한 분할, text-embedding-3-small, GPT-5 mini를 사용했다. 모델 직접 실행 방식은 고정 길이 분할에 짧은 사업 개요 조각을 추가하고 BGE-M3와 EXAONE-3.5-7.8B-Instruct를 사용했다.

원본은 HWP 96건과 PDF 4건으로 총 100건이며 모두 텍스트를 추출했다. 중복 제거 후 최종 평가는 두 방식이 같은 98개 문서와 질문 50개를 사용했다. 질문은 답이 있는 42개와 답이 없는 8개로 구성했다. 각 방식의 3회 평가에서 정답 문서 검색 지표는 모두 1.0이었으며, 답변 점수 평균은 API 활용 방식 0.996, 모델 직접 실행 방식 0.849였다. 답이 없는 질문의 거절은 각각 8/8과 7/8이었다. 이 점수는 답이 있는 42문항에 부분 점수를 적용한 자동 채점 평균이다.[1][2]

이번 평가에서는 API 활용 방식의 답변 점수와 응답 시간이 더 좋았다. 두 방식의 구성 요소가 함께 반영된 결과이며, 평가 범위와 자동 판정의 변동은 결과 분석에서 함께 다룬다. 운영 비용은 실제 청구액이 아니라 월 720시간과 질문 1,000회를 가정한 부분 비용 예시로 산정했다. CPU 서버를 사용하는 API 운영 구성은 별도의 성능 검증이 필요하다.

# 목차

1. 프로젝트 개요
2. 데이터 분석과 문서 전처리
3. 시스템 설계와 초기 구현
4. 실험 및 평가 설계
5. API 활용 방식의 비교 실험과 선택 근거
6. 모델 직접 실행 방식의 비교 실험과 선택 근거
7. 최종 시스템 구성
8. 공통 조건의 최종 평가 결과
9. 독립 운영을 가정한 비용 분석
10. 성과, 한계 및 향후 과제
참고자료
부록 1. 실험 조건과 근거 대응표
부록 2. 실행 및 재현 절차

# 1. 프로젝트 개요

## 1.1 추진 배경과 업무 문제

공공입찰 검토에서는 사업 요구사항뿐 아니라 참여 자격, 사업 예산과 기간, 제안서 제출 조건 등을 함께 확인해야 한다. 숫자만 맞더라도 예외 조건이나 적용 대상을 빠뜨리면 검토 결과가 달라질 수 있다. 필요한 정보가 표와 본문 여러 곳에 나뉘어 있어, 사용자가 답변을 확인한 뒤 원문으로 돌아가 근거와 조건을 다시 검토할 수 있어야 한다.

본 프로젝트는 공공입찰 컨설팅 서비스 '입찰메이트'의 사내 업무 지원을 주제로 한다. 문서를 직접 읽는 검토 과정을 보조하기 위해 질문과 관련된 내용을 찾아 정리하고, 확인 가능한 원문 출처를 함께 제공한다. 사용자는 제시된 답변과 원문 근거를 검토해 입찰 판단에 활용한다.

## 1.2 프로젝트 목표와 구현 범위

첫째, 원본 입력부터 텍스트 추출, 문서 분할, 의미 표현 생성, 검색, 답변 생성과 평가까지 이어지는 실행 가능한 흐름을 확보한다. 둘째, 질문에 필요한 근거를 답변 모델에 전달하고 출처 번호로 연결한다. 셋째, 여러 모델과 설정을 동일한 비교 조건 안에서 검토하여 선택 이유와 한계를 남긴다.

전체 기획에는 정보 검색, 핵심 내용 요약, 질의응답과 대화 맥락 유지가 포함됐다. 최종 보고서의 정량 검증 범위는 저장 문서에 대한 단일 질문 기반 질의응답이다. 다회차 대화와 사용자 만족도, 업무 시간 절감은 후속 검증 과제로 정리했다.

## 1.3 두 구현 방식의 추진 목적

API 활용 방식은 외부 임베딩·답변 모델을 연결하여 전체 질의응답 흐름을 구현하고 문서 처리와 검색·근거 전달 조건을 개선하는 데 초점을 두었다. 모델 직접 실행 방식은 서버에 모델을 올리고 GPU 메모리, 패키지 호환성과 응답 속도를 다루며 같은 업무 목적을 구현했다. 각 방식에서 여러 모델과 설정을 비교한 뒤 최종 구성을 정했다.

두 방식 모두 검색과 프로그램 실행을 위한 서버를 사용한다. API 활용 방식도 검색 결과를 다시 평가하는 모델은 서버에서 실행한다. 두 방식은 임베딩·답변 모델의 실행 위치와 이에 따른 운영 책임에서 차이가 있다.

## 1.4 개발 단계와 역할

개발은 초기 전체 흐름 연결, 데이터 분석과 전처리 개선, 평가셋 확장, 모듈별 비교 실험, 방식별 통합과 공통 조건 평가 순으로 진행했다. 초기 베이스라인은 데이터 분석에 앞서 연결 가능한 출발점을 마련했고, 이후 관찰한 문서 특성과 오류 사례를 개선에 반영했다.

표 1. 주요 역할

| 담당자 | 주요 역할 |
| --- | --- |
| 김도영 | PM·통합, 초기 흐름 구성과 청킹 비교 |
| 나상훈 | 파싱·전처리, API 활용 방식 통합 개선과 재현 평가 |

| | |
|---|---|
| 유찬혁 | 청킹·임베딩, 직접 실행 모델 비교와 개요 추가 검증 |
| 김연주 | 검색 고도화, 후보 검색·재정렬·검색 진단 |
| 박단비 | 생성 모델·지시문, 모델 직접 실행과 GPU 안정화 |
| 김시현 | 평가 데이터·지표·채점 기준 |

담당 구분은 개발과 개선의 주된 책임을 나타낸다. 통합 과정에서는 여러 담당자가 같은 흐름과 결과를 교차 확인했다. 공통 입출력을 유지하면서 내부 구현을 바꾸고, 평가 기록과 코드 버전을 함께 남기는 원칙을 적용했다.[3][4]

# 2. 데이터 분석과 문서 전처리

## 2.1 원본 문서와 메타데이터

제공 원본은 HWP 96건과 PDF 4건으로 총 100건이다. CSV에는 파일명, 사업명, 발주 기관 등 메타데이터와 일부 본문이 포함됐다. CSV 본문은 문서 앞부분에서 끊긴 경우가 있어 전체 검색 자료로 사용하기에 부족했다. 이에 원본 전체에서 본문을 확보하고 CSV는 문서와 사업 정보를 연결하는 데 활용했다.

중간 분석 기록에서 추출 후 문서당 평균 분량은 HWP 약 8.1만 자, PDF 약 12.7만 자로 보고됐다. 표 내부 문자 비중은 각각 약 67%와 61%였다. 이 수치는 추출 텍스트의 문자량을 기준으로 표 중심 문서의 특성을 보여준다. 세부 집계 방식의 보완은 후속 데이터 분석 과제로 남겼다.[3]

## 2.2 표 중심 문서의 처리 과제

예산, 일정, 요구사항과 평가기준은 표 안에 자주 나타난다. 표를 단순히 줄 단위 텍스트로 풀면 어떤 값이 어느 항목에 속하는지 알기 어려워진다. 표 안의 문자를 확보하는 일과 항목·값의 대응 관계를 유지하는 일을 함께 처리해야 했다.

CSV 금액과 원문 금액을 대조한 초기 기록에서는 그대로 일치한 문서가 69%였고 0원으로 기재된 문서도 6건 있었다. 불일치 사례에는 단위와 부가가치세 포함 여부의 차이가 있었다. 이에 예산 정답은 원문 표기와 단위, 세금 조건을 확인해 정하는 원칙을 적용한다.[3]

## 2.3 본문과 표 추출의 개선

초기 HWP 처리에는 기존 텍스트 추출 기능을 사용했으나 표 내부 내용 누락이 관찰됐다. 최종 코드는 HWP 내부의 문단과 표 기록을 읽고 셀의 행·열·병합 정보를 이용해 텍스트 표로 복원한다. 셀 안에 중첩된 표는 바깥 표를 깨뜨리지 않도록 한 줄 표현으로 정리한다.

PDF는 최종 코드에서 PyMuPDF를 사용한다. 표 영역을 찾아 셀을 추출하고, 표 밖의 본문은 읽기 순서를 고려해 결합한다. 표 안의 글자를 본문에서 다시 넣지 않도록 제외하며, 쪽번호와 장식용 테두리·머리말을 분리한다. 페이지를 넘겨 이어지는 표를 연결하는 처리도 포함한다. 초기 pypdf 기반 처리에서 표 구조 복원을 지원하는 최종 구현으로 확장했다.[4]

## 2.4 텍스트 정제와 문서 구조 정보

‘사 업 명’처럼 자간이 벌어진 제목·항목을 결합하고, 불필요한 제어 문자와 쪽번호 등을 정리했다. 파일명은 한글의 같은 글자가 서로 다른 문자 조합으로 저장되는 차이를 정규화하여 연결 오류를 줄였다. 이 과정을 통해 원문 내용을 검색하기 좋은 형태로 정리했다.

절 제목과 붙임·별첨·서식 구간을 구분하고 본문 위치와 함께 저장했다. 구조 정보는 문서를 나눌 때 경계를 지키거나 검색 후보를 평가할 때 맥락을 제공하는 데 사용했다. 청크는 검색을 위해 나눈 문서 조각이며, 각 조각에 원문 문서 식별자와 정제 본문 안의 위치를 유지했다.

## 2.5 100건 추출과 최종 98개 문서

표 복원과 정제 후 원본 100건 모두에서 텍스트를 추출했다. 이후 정제 본문이 완전히 같은 문서를 중복으로 묶고, 공고 번호가 있는 문서를 우선 남기는 방식으로 최종 98개 문서를 확정했다. 최종 평가는 두 방식이 동일한 문서 파일을 사용해 데이터 조건을 맞췄다.[2][4]

초기 베이스라인에도 98개 문서라는 수치가 등장하지만 원본 100건 중 2건 추출 실패로 남은 수치다. 최종 98개는 100건 추출 후 본문 중복을 제거한 결과이므로 데이터 구성과 처리 상태가 다르다. 최종 코드의 오류 기록 파일은 추출 실패와 중복 제외 사유를 함께 기록하며, 처리 결과는 사유별로 집계한다.

# 3. 시스템 설계와 초기 구현

## 3.1 전체 처리 흐름

문서 준비 단계는 원본 본문·표 추출 → 정제·메타데이터 결합 → 문서 분할 → 임베딩 생성 → 검색 인덱스 저장 순으로 구성한다. 임베딩은 글의 의미를 비교하기 위한 숫자 배열이다. 미리 생성한 문서 임베딩과 원문 조각을 저장하여 질문마다 전체 문서를 다시 처리하지 않도록 했다.

질문 처리 단계는 질문 임베딩 → 의미·키워드 후보 검색 → 후보 순위 결합 → 질문과의 관련성 재평가 → 최종 근거 선택 → 답변·출처 생성 순으로 구성한다. RAG는 이렇게 필요한 문서 내용을 먼저 찾고, 그 내용을 바탕으로 답변을 만드는 방식이다.

표 2. 주요 데이터와 역할

| 데이터 | 주요 내용 | 역할 |
|---|---|---|
| 문서 | 문서 식별자, 정제 본문, 메타데이터, 구조 정보 | 원본과 검색 자료를 연결한다. |
| 문서 조각 | 조각 식별자, 본문, 소속 문서와 위치 | 검색할 단위를 만든다. |
| 검색 결과 | 문서 조각과 검색 점수 | 답변에 전달할 근거를 고른다. |
| 답변 | 질문, 답변, 출처 목록, 모델과 상태 | 결과를 검토·평가·재현한다. |

## 3.2 원문과 답변 출처의 연결

문서 조각은 소속 문서와 정제 본문 내 문자 위치를 보유한다. 검색 결과에 출처 번호를 붙여 답변 모델에 제공하고, 답변에는 [1]과 같은 번호로 근거를 표시하도록 했다. 이 위치는 정제된 본문 안에서 해당 조각을 찾아가는 데 사용한다.

결과 파일의 출처 목록은 모델에 제공한 검색 자료를 기록한다. 출처 목록에 문서가 있다는 사실과 답변 문장이 실제로 그 문서를 인용했다는 사실은 다르다. 평가에서는 답변에 사용된 번호가 정답 문서를 가리키는지 따로 확인한다.

## 3.3 초기 베이스라인

초기 베이스라인은 고정 길이 1,000자와 중복 150자로 문서를 나누고, text-embedding-3-small로 의미 표현을 만든 뒤 벡터 검색 상위 5개를 GPT-5 mini에 전달했다. 메타데이터 필터와 결과 저장, 평가 함수 연결을 포함하여 전체 흐름이 동작하는 출발점을 마련했다.

초기 베이스라인의 65문항 평가에서 Recall@5는 0.7787, MRR@5는 0.7633, 답변 키워드 포함률은 0.4282, 평균 응답 시간은 약 3.70초였다. 이 평가는 추출 성공 98개 문서와 초기 65문항 평가셋을 사용했다. 기관명 표기 불일치로 검색이 비어 있던 3문항도 평균에 포함됐다.[5]

초기 지표로 기관명 필터 불일치와 근거 검색 문제를 확인하고 후속 개선 방향을 정했다. 이후 문서·질문·채점 기준을 확장해 최종 평가를 구성했다. 키워드 포함률은 기대 표현의 포함 정도를 확인하는 보조 지표로 활용했다.

## 3.4 통합과 재현 구조

최종 통합 과정에서는 문서 처리, 평가, 실험 기록과 질문 재작성 등 두 방식이 공유하는 기능을 공통 모듈로 정리했다. 임베딩·검색·생성 및 실행 진입점은 각 방식의 차이를 유지한다. 실행기는 방식별 가상환경의 Python을 선택하고, 두 방식을 선택하면 차례로 실행해 평가 요약을 보여준다.

실행 시 전처리 문서와 인덱스에 저장한 문서 해시를 비교한다. 해시는 파일 내용을 식별하는 값으로, 다른 문서로 만든 인덱스를 섞어 쓰는 문제를 막는 데 사용한다. 문서 분할이나 임베딩 모델을 변경하면 새 인덱스를 구축하고, 답변 모델만 바꾸는 경우에는 같은 인덱스를 재사용할 수 있다.[4]

# 4. 실험 및 평가 설계

## 4.1 선택 실험과 최종 평가

선택 실험은 특정 모델이나 설정을 정하기 위한 비교이며, 최종 평가는 각 방식에서 선택한 구성을 공통 데이터로 확인하는 과정이다. 모델 후보 비교, 검색 전용 실험, 저장 답변 재채점과 전체 재실행을 구분했다. 검색 전용 실험은 후보와 근거 확보를 평가하고, 전체 실행 실험은 답변 품질과 응답 시간을 함께 평가했다.

표 3. 대표 평가 조건의 구분

| 목적 | 데이터·질문 | 해석 범위 |
|---|---|---|
| 초기 기준선 | 추출 성공 98개, 초기 46·65문항 | 초기 흐름과 오류를 확인한다. |
| 임베딩 모델 선택 | 100개 문서, 65문항, 의미 검색만 | BGE-M3와 KURE-v1을 비교한다. |
| API 답변 모델 선택 | 356문항, 인용 평가 350문항 | mini와 nano의 답변·출처·비용을 비교한다. |
| 청킹·검색 탐색 | 실험별 100개 또는 98개, 평가셋별 상이 | 같은 실행 조건 안에서 설정을 비교한다. |
| 직접 실행 답변 모델 선택 | 저장 검색 자료, 이전 50문항 | EXAONE과 Qwen2.5를 비교한다. |
| 최종 공통 평가 | 동일 98개, 수정 50문항, 각 3회 | 선택한 두 구현의 품질과 시간을 확인한다. |

## 4.2 최종 평가셋

최종 평가셋은 질문과 참고 근거를 검수한 50개 문항으로 구성했다. 문서에 답이 있는 질문 42개와 답을 확인할 수 없는 질문 8개를 포함한다. 기간과 금액 같은 수치, 참가 자격과 평가 방법 같은 설명형 질문, 근거 부족 상황의 거절을 함께 확인한다. 사용한 평가셋의 식별 정보는 부록의 근거표에 정리했다.

같은 50문항이라는 이름을 가진 이전 평가셋도 일부 참고 근거와 질문이 수정돼 파일 해시가 다르다. 각 버전의 결과는 해당 질문과 참고 근거에 맞춰 정리했다. 추가 평가셋 후보도 탐색했으나 검증이 완료되지 않은 확장 34문항은 최종 비교에서 제외했다.[2]

## 4.3 검색과 근거 평가

Recall@5는 상위 검색 결과에서 찾은 정답 문서 수를 해당 질문의 전체 정답 문서 수로 나눈 값의 평균이다. 정답 문서가 여러 개면 일부만 찾아도 부분 값을 갖는다. MRR@5는 상위 5개 검색 조각 안에서 첫 정답 문서가 나타난 순위의 역수를 평균한다. 같은 문서의 조각을 제거한 문서 순위와는 다르다.

근거 적중률은 해당 평가 정보를 가진 질문에서 정답 문서의 숫자·키워드 단서가 검색 결과에 포함됐는지 확인하는 보조 지표다. 대표 청킹 비교에서는 20문항을 대상으로 했다. 정답 문서를 찾았더라도 필요한 문단이 빠질 수 있어 문서 검색과 분리해 확인했다. 근거 단서와 실제 답변을 함께 평가해 검색과 생성 단계의 문제를 나눠 확인했다. 최종 기록의 근거 지표는 저장 출처를 기준으로 집계하며, API 활용 방식에서는 생성에 추가한 다음 조각도 포함한다.

## 4.4 답변·출처·거절 채점

수치형은 단위와 표기를 정규화한 값으로, 키워드형은 기대 항목의 포함 여부로 비교한다. 설명형은 GPT-5 nano 판정기로 정답·부분 정답·오답을 구분하여 1점·0.5점·0점을 부여한다. 최종 답변 점수는 답이 있는 42문항의 평균이며, 출처가 필수인 문항에서 정답 문서를 인용하지 않으면 내용이 맞더라도 점수를 제한한다.

내용 점수는 출처 규칙을 적용하기 전 내용만 본 결과다. 정답 문서 인용률은 답변의 출처 번호가 정답 문서를 가리키는지 확인한다. 문장별 사실성과 조건의 완전성은 원문 대조를

통해 보완한다. 답이 없는 8문항은 거절 성공을 별도로 집계하고, 답이 있는 질문을 잘못 거절한 경우도 분리한다.

최종 판정기는 핵심 항목의 누락과 원문과의 모순을 확인하는 기준을 적용했다. 참고 근거가 발췌본이라는 점을 반영하여, 발췌 밖의 추가 설명이라는 이유만으로 감점하지 않고 모순·다른 사업의 내용과 핵심 항목 누락을 확인하도록 했다. 재채점 결과에는 변경한 판정 기준을 함께 기록했다.[2][4]

## 4.5 반복과 시간 측정

각 방식은 같은 50문항으로 3회 평가하고, 회차마다 판정 캐시 없이 새로 채점했다. 동일한 50문항의 반복 실행으로 자동 판정의 변동과 실행 시간을 관찰했다.

전체 응답 시간은 질문 처리 함수를 호출한 시점부터 답변이 반환될 때까지 측정한다. 질문 임베딩, 검색·재정렬과 답변 생성을 포함한다. 문서 준비·인덱스 구축·모델 로딩은 사전 준비 단계로, 채점·파일 저장은 후처리 단계로 분리했다.

P50은 응답 시간의 중간 구간, P95는 느린 응답 구간을 보여주는 분위수다. 최종 표의 P50·P95는 회차별 분위수를 구한 뒤 세 값을 평균한 결과다. 검색만 측정한 시간이나 다른 동시성 조건의 실험 시간은 최종 응답 시간과 분리한다.

# 5. API 활용 방식의 비교 실험과 선택 근거

## 5.1 문서 분할 방식과 길이

고정 길이 분할과 제목·절·표 경계를 고려한 구조 기반 분할을 비교했다. 초기 작은 평가셋에서는 문서 검색 지표가 이미 높아 우열을 가리기 어려웠다. 이후 본문 세부 질문을 추가해 검색 근거와 답변, 입력 분량을 함께 확인했다. 구조 기반 분할은 표·절의 연결 유지와 입력 분량 감소에 이점을 보였고, 검색 조건에 따른 차이는 후속 실험에서 확인했다.

최종 길이 선택에는 98개 문서와 이전 50문항 버전의 비교 실험을 사용했다. 구조 기반 분할의 길이 700·1,000·1,500자와 중복 100·150·200자를 비교했다. 모든 조건의 검색 결과를 평가하고, 700자와 기준 1,000자는 전체 답변까지 비교했다.

표 4. API 활용 방식의 길이·중복 비교

| 길이·중복 | 문서 **Recall@5 / MRR@5** | 상위 **5**개 근거 단서 | 답변 평가와 판단 |
|---|---|---|---|
| 1,000 / 150 | 1.000 / 1.000 | 1.00 | 답변 점수 0.9603, 기준 유지 |
| 700 / 150 | 1.000 / 0.9810 | 0.95 | 답변 점수 0.9167, 채택 보류 |
| 1,500 / 150 | 1.000 / 0.9841 | 1.00 | 검색·문맥 분량 증가, 답변 미측정 |
| 1,000 / 100 | 1.000 / 0.9821 | 1.00 | 뚜렷한 이점 부족, 답변 미측정 |
| 1,000 / 200 | 1.000 / 0.9810 | 0.95 | 중복 증가의 개선 근거 부족, 답변 미측정 |

700자는 생성 입력을 약 16% 줄이고 평균 응답도 짧아졌지만 답변 점수가 낮아졌다. 실제 캐시 할인까지 반영한 질문당 계산 비용은 기준보다 약 2.8% 높았다. 1,500자는 근거 단서의 추가 이점 없이 검색 시간과 문맥 분량이 커져 전체 답변 후보에서 제외했다. 이 조건에서는 1,000자·중복 최대 150자를 유지했다. 표 4는 이전 평가셋과 코드에서 분할 설정을 선택한 결과이며, 최종 평가 결과는 8장에 제시한다.[2: 46~50행]

## 5.2 의미 검색과 키워드 검색의 결합

의미 검색은 표현이 달라도 비슷한 내용을 찾는 데 사용하고, BM25 키워드 검색은 사업명이나 특정 항목의 어휘를 찾는 데 사용했다. 두 검색의 상위 순위를 결합해 후보를 만들었다. 점수 크기가 다른 두 모델의 값을 직접 더하는 대신 순위 기반 결합 방식을 적용했다.

초기 개인 검색 평가에서는 여러 설정을 함께 적용한 결과 문서 검색 지표가 좋아졌지만, 평가셋이 메타데이터로 자동 생성됐고 팀 공통 평가와 달랐다. 이후 공통 문서와 질문으로 후보 검색, 재정렬과 문서별 선택 제한을 나눠 확인했다. 검색 후보에 사업명·발주 기관 정보를 붙여 문서 식별을 보완했다.

## 5.3 후보 확대와 관련성 재평가

질문과 문서 조각을 함께 읽어 질문과의 관련성을 평가하는 BAAI/bge-reranker-v2-m3를 사용했다. 단순 단어 겹침 중심 재정렬과 비교했을 때 필요한 본문 조각을 상위로 올리는 데 더 나은 결과를 보였다. 다만 정답 조각이 처음 후보에 없으면 재정렬만으로 복구할 수 없었다.

이 문제를 확인하기 위해 공통 65문항과 별도의 문서 내부 질문 19개를 사용했다. 19문항에서 후보 확대 전 근거 적중은 약 0.74였고, 상위 문서 안의 키워드 후보를 더 넣으면 약 0.95로 높아졌다. 추가 후보 수 5·10·20에서 결과가 같아 가장 작은 5개를 선택했다. 이 결과를 바탕으로 후보 확대 설정을 선택했다.[6]

최종 검색은 의미 검색 100개와 키워드 검색 100개의 순위를 결합한 뒤 상위 50개를 기본 재정렬 후보로 사용한다. 상위 2개 문서에서 각 최대 5개 후보를 추가할 수 있어 재정렬 입력이 최대 60개로 늘어난다. 이는 최종 답변에 넘기는 검색 결과 5개와 구분한다.

## 5.4 질문 의도와 문서별 제한

사업 설명이 길게 붙은 질문에서는 모델이 사업 전체 설명에 집중하고 실제로 묻는 항목을 놓치는 문제가 있었다. 질문에서 사업명·발주 기관을 덜어낸 나머지 항목이나 여러 문장 중 실제 질문 부분을 이용해 후보를 보완하고, 1위 문서 안에서 질문 항목에 맞춰 순서를 조정했다.

기본적으로 한 문서의 조각이 결과를 독점하지 않도록 문서당 최대 2개를 선택한다. 그러나 특정 사업 하나에 대한 질문은 필요한 조각이 세 번째 이후에 남아 제한으로 사라지는 문제가 있어 해당 질문에서 제한을 해제했다. 여러 사업을 명시적으로 비교하는 질문은 대상 문서의 후보를 함께 재평가해 한 사업에만 치우치지 않도록 했다.[2][4]

질문을 별도 모델로 다시 쓰는 방식도 비교했으나 문서 내부의 특정 항목을 찾는 성능이 낮아지고 추가 호출과 지연이 발생해 기본 구성에서 제외했다. 개별 평가셋에 맞춘 질문

처리의 위험을 확인하기 위해 표현을 바꾼 질문과 추가 문서 질문도 탐색했다. 일부 변형에서 나타난 성능 저하는 질문 표현에 대한 대응을 보완할 과제로 정리했다.

## 5.5 근거 전달과 답변 지시문

참가 자격 목록이 검색 조각의 끝에서 끊겨 일부 요건이 답변에 빠지는 사례를 확인했다. 선택한 1위 조각의 바로 다음 조각을 같은 문서에서 찾아 답변 생성 직전에 추가했다. 다음 조각이 없거나 이미 선택돼 있으면 추가하지 않는다. 검색 결과는 기본 5개지만 실제 생성 입력은 필요에 따라 6개가 될 수 있다.

해당 개선 실험에서 참가 자격 문항은 3회 모두 정답으로 바뀌었으나 전체 평균은 판정 변동과 겹쳤다. 이 결과로 목록이 다음 조각에 이어지는 사례의 누락 보완 효과를 확인했다. 선택한 검색 결과와 생성에 추가한 다음 조각은 결과 저장 시 별도 표시한다.

지시문은 결론을 먼저 쓰고 사업 간 정보를 섞지 않으며, 근거로 확인한 문장에만 출처 번호를 붙이도록 정리했다. 검색 조건이 명시된 질문에서 불필요하게 사업을 되묻지 않도록 했고, 같은 문서의 붙임 표에서 사업명 표기가 다를 때에는 그 차이를 밝히도록 했다. 답변에는 문서에서 확인한 값과 표기 차이를 함께 전달하도록 했다.

## 5.6 답변 모델 선택

GPT-5 mini와 GPT-5 nano를 비교했다. 초기 65문항에서는 차이가 작았으나 이후 356문항으로 재확인했다. 이 비교에는 당시의 단어 겹침 기반 재정렬을 사용했다. 인용은 정답 문서 기준이 있는 350문항, 시간·계산 비용은 전체 356문항을 기준으로 집계했다.

표 5. API 답변 모델 선택 실험

| 항목 | **GPT-5 mini** | **GPT-5 nano** |
|---|---|---|
| 정답 문서 인용 / 350문항 | 341 | 326 |
| 평균 응답 시간 / 356문항 | 5.00초 | 4.83초 |
| 계산 비용 / 356문항 | 0.889달러 | 0.181달러 |
| 판단 | 출처 제공을 우선해 채택 | 비용 절감 대안으로 유지 |

nano는 시간과 계산 비용에서 이점이 있었지만 mini가 정답 문서를 인용한 문항이 15개 많았다. 본 서비스는 답과 원문 근거를 함께 제공하는 목적이 있어 mini를 선택했다. 키워드 점수에 영향을 주는 금액 표기 차이도 확인하고, 출처 제공을 주요 선택 기준으로 삼았다. 표 5의 비용은 기록된 사용량과 단가로 계산했다.[1][2][7]

# 6. 모델 직접 실행 방식의 비교 실험과 선택 근거

## 6.1 실행 환경과 GPU 자원

모델 직접 실행 방식은 GCP Linux의 NVIDIA L4 환경에서 임베딩, 재정렬과 답변 생성을 연결했다. 임베딩·재정렬·생성 모델을 한 GPU에서 함께 실행하며 통합 안정성을 확인했다. 로딩한 모델을 재사용하고 GPU 메모리와 오류를 점검했다.

EXAONE은 최종 구현에서 bfloat16으로 GPU에 적재하고, 매 단계 가장 높은 확률의 토큰을 고르는 방식으로 생성했다. 출력 상한은 1,024토큰이다. 서로 다른 패키지 버전이 필요한 두 방식을 별도 가상환경으로 분리하고 최신 실행기에서 각각의 환경으로 실행하도록 했다.[4]

## 6.2 임베딩 모델 비교

BGE-M3와 KURE-v1은 같은 100개 문서, 65문항, 고정 길이 1,000자·중복 150자, 10,218개 조각 조건에서 비교했다. 의미 검색만 사용하고 키워드 검색과 재정렬은 끈 상태로 모델별 3회 측정했다. 평가 범위는 질문 임베딩과 검색 결과로 정했다.

표 6. 직접 실행 임베딩 모델 비교

| 모델 | **Recall@5** | **MRR@5** | 평균 검색 시간 |
|---|---|---|---|
| BGE-M3 | 0.9351 | 0.9513 | 35.04ms |
| KURE-v1 | 0.9262 | 0.9628 | 35.36ms |

정답 문서를 빠뜨리지 않는 비율을 우선해 BGE-M3를 선택했다. KURE-v1은 첫 정답의 순위에서 더 좋은 결과를 보였다. 두 지표의 특성을 비교해 문서 검색률을 우선했으며, 이 결과를 의미 검색 전용 조건의 모델 선택 근거로 활용했다.[8]

## 6.3 문서 분할과 문맥 보완 비교

일정 길이 분할, 문단·문장 경계 분할과 구조 기반 분할을 비교하고, 길이 500·1,000·1,500자도 검토했다. 의미 검색 조건에서 작은 조각은 문서 검색에, 큰 조각은 근거 단서 포함에 이점을 보였다. 검색 설정이 바뀌면 조각 선택 결과도 달라져 최종 검색 조건에서 다시 확인했다.

사업명·기관을 본문 앞에 붙이거나 본문 임베딩과 문맥을 붙인 임베딩을 섞는 방식도 비교했다. 의미 검색에서 일부 이점이 있었던 혼합 방식은 최종 키워드 검색·재정렬 결합 조건에서 기준선의 문서 검색·근거 적중을 개선하지 못했다. 두 종류의 문서 임베딩을 새로 만드는 부담을 추가할 근거가 부족해 최종적으로 본문 중심 임베딩을 유지했다.[9]

구조 기반 분할은 일부 조건에서 문서 Recall을 높였으나 문서당 2개 제한으로 필요한 세 번째 조각이 사라지는 사례가 있었다. 원문 조각을 확인한 결과 근거는 보존돼 있었고, 누락은 최종 선택 제한 단계에서 발생했다. 이에 문서 분할과 후보 선택·제한을 함께 점검했다.

## 6.4 사업 개요 추가와 최종 선택

최종적으로 기존 고정 길이 조각을 모두 유지하고, 짧은 사업 개요를 별도 검색 조각으로 더하는 방식을 선택했다. 개요는 원문에서 사업명·기간·예산·계약 방식 중 두 종류 이상을 포함하는 짧은 구간을 추출해 구성했다. 개요 정보를 포함한 원문 자체는 두 방식 모두 보유하며, 차이는 별도 검색 단위의 구성에 있다.

표 7. 개요 추가 선택 실험

| 조건 | 문서 수·조각 수 | **Recall@5 / MRR@5** | 근거 적중 **/ 20**문항 |
|---|---|---|---|

| | | | |
|---|---|---|---|
| 고정 길이만 | 100개 / 10,218개 | 0.9762 / 0.9762 | 18/20 = 0.90 |
| 고정 길이 유지와 개요 추가 | 100개 / 10,266개 | 0.9762 / 0.9762 | 19/20 = 0.95 |

고정 길이 조각을 보존하면서 48개 개요 조각을 더했을 때 기간 근거 한 문항을 추가 확보했고 문항별 검색 지표 하락은 없었다. 기존 벡터를 재사용한 비교뿐 아니라 일반 빌드에서 전체를 다시 임베딩한 비교에서도 검색 결과가 재현됐다. 100개 문서 조건의 별도 답변 비교에서는 점수가 0.798에서 0.821로 높아졌다. 이후 최종 공통 98개 문서 조건에서 전체 구성을 평가했다.[10]

최종 공통 평가를 위해 동일한 98개 문서로 다시 구축한 인덱스는 10,008개 조각이다. 100개 문서 실험의 10,266개와 구분한다. 추가 개요도 검색 후보와 문서별 선택 대상에 포함하므로, 최종 선택 결과에서 기존 근거가 유지되는지 함께 확인한다.

## 6.5 답변 모델과 지시문 비교

EXAONE-3.5-7.8B-Instruct와 Qwen2.5-7B-Instruct를 같은 저장 검색 자료와 지시문으로 비교했다. 이전 50문항 조건에서 Qwen의 키워드 포함률은 높았지만 답변에 출처 번호가 빠진 경우가 많았다.

표 8. 직접 실행 답변 모델 선택 실험

| 항목 | **EXAONE** | **Qwen2.5** |
|---|---|---|
| 키워드 포함률 | 0.750 | 0.825 |
| 정답 문서 인용률 / 답 있는 42문항 | 0.976 | 0.190 |
| 답 없는 질문 거절 / 8문항 | 7/8 | 5/8 |
| 평균 응답 시간 | 12.72초 | 15.65초 |

원문 출처 제공과 거절을 우선해 EXAONE을 선택했다. 다만 지시문은 EXAONE을 중심으로 조정했으므로 다른 모델에 불리할 수 있다. 이 비교는 EXAONE 중심으로 조정한 공통 지시문에서 출처 제공과 거절을 확인한 결과다. 최종 구성의 평가는 8장에 정리한다.[1][2: 79~80행]

지시문에는 낙찰 금액·낙찰 업체처럼 공고 이후 정보는 추측하지 않고, 여러 항목을 물으면 빠짐없이 답하며, 수치를 원문 표기대로 유지하는 규칙을 추가했다. 긴 근거 뒤에서 질문을 놓치는 문제를 줄이기 위해 입력 끝에 질문을 다시 제시했다.

## 6.6 검색·근거 전달과 안정화

재정렬 입력에 절 제목을 붙여 같은 문서에서 어떤 항목을 다루는지 함께 평가했다. 문서와 관련이 낮은 질문은 의미 유사도 기준으로 차단하는 조건도 비교했다. 최종 기준은 직접 실행 임베딩 모델의 점수 분포에 맞춰 0.53으로 정했다. API 활용 방식은 해당 임베딩 모델에 맞춘 0.40을 사용한다.

다음 조각 추가와 특정 사업 질문의 문서별 제한 해제 등 API 활용 방식에서 사용한 개선도 직접 실행 방식에서 검토했다. 최종 구성은 직접 실행 환경의 평가 결과에 따라 절 제목 활용, 문서당 최대 2개와 선택 결과 5개로 정했다.

4비트 압축 실행은 메모리 사용량을 줄였지만 응답 시간의 뚜렷한 개선과 거절 품질 유지가 함께 확인되지 않아 채택하지 않았다. 답변을 더 짧게 쓰도록 한 실험도 인용과 거절 결과가 낮아져 기본 구성에서 제외했다. 다른 압축 모델은 패키지 호환 문제로 실행하지 못한 항목을 실측 결과와 구분했다.

GPU 메모리가 잘게 나뉘어 남는 문제를 줄이는 할당 설정을 적용한 실험에서는 메모리 부족 경고가 1회에서 0회로, 관찰된 최대 사용량이 22.4GB에서 21.5GB로 줄었다. 점수 향상보다 실행 안정화의 근거로 채택했다. 이는 해당 50문항 실행에서 관찰한 최대 사용량이다.[2: 81~88행]

# 7. 최종 시스템 구성

## 7.1 공통 구성과 차이

표 9. 최종 구성 비교

| 항목 | **API** 활용 방식 | 모델 직접 실행 방식 |
|---|---|---|
| 최종 검색 문서 | 동일 98개 | 동일 98개 |
| 문서 분할 | 제목·절·표 구조 반영,<br>1,000자·중복 최대 150자 | 고정 1,000자·중복 150자와 원문 사업 개요 추가 |
| 검색 조각 수 | 11,551개 | 10,008개 |
| 임베딩 | text-embedding-3-small,<br>1,536차원 | BGE-M3, 1,024차원 |
| 후보 검색 | 의미 검색·키워드 검색 결합 | 의미 검색·키워드 검색 결합 |
| 재정렬 모델 | BAAI/bge-reranker-v2-m3 | BAAI/bge-reranker-v2-m3 |
| 질문별 보완 | 묻는 항목 재정렬, 비교 사업 후보 보완 | 본문과 절 제목 함께 평가 |
| 문서별 선택 제한 | 기본 2개, 특정 단일 사업 질문에서 해제 | 최대 2개 |
| 생성 입력 | 검색 결과 5개와 필요시 다음 조각 | 검색 결과 5개 |
| 관련성 차단 | 해당 임베딩 유사도 기준 0.40 | 해당 임베딩 유사도 기준 0.53 |
| 답변 모델 | GPT-5 mini | EXAONE-3.5-7.8B-Instruct |

최종 구성은 같은 목적을 가진 두 구현의 비교다. 평가 결과에는 임베딩, 문서 분할, 검색 보완과 생성 입력이 함께 반영된다. API 활용 방식은 임베딩·답변 모델을 API로 호출하고 검색·재정렬은 서버에서 수행한다.

## 7.2 저장 구조와 실행 관리

원본과 공통 전처리 문서, 방식별 인덱스와 결과를 구분한다. 인덱스에는 벡터, 조각 본문·메타데이터와 설정을 저장한다. 결과에는 답변과 출처, 모델, 검색 설정,

문서·평가셋·인덱스 식별값, 코드 버전과 실행 상태를 함께 남긴다. 토큰 사용량과 판정 사용량도 구분한다.

최종 통합 코드에서는 두 방식이 같은 전처리 자료를 사용하도록 저장 구조를 정리했다. 각 실험에 사용한 문서와 인덱스를 함께 보관하고, 재현할 때에는 파일 내용의 식별값을 대조해 동일한 입력 조건을 확보한다.

## 7.3 최종 통합과 성능 측정의 연결

보고서의 구현 설명에는 공통 모듈 정리와 방식별 실행 환경 선택을 반영한 최종 통합 구성을 사용했다. 성능 수치는 그에 앞서 완료한 두 방식의 공통 조건 3회 평가를 기준으로 정리했다. 구현과 측정에 사용한 코드 버전 및 실험 기록은 부록의 근거표에서 연결한다.

API 활용 방식은 최종 구성과 같은 처리 코드로 재현 평가를 수행했고, 모델 직접 실행 방식은 공통 98개 문서로 인덱스를 다시 구축해 평가했다. 공통 모듈 정리 이후의 코드도 같은 입력의 출력과 지표가 유지되는지 확인하는 절차로 관리한다.[2][4]

# 8. 공통 조건의 최종 평가 결과

## 8.1 최종 평가 조건

두 방식 모두 중복 제거 후 동일한 98개 문서와 검수한 평가 질문 50개를 사용했다. 임베딩 모델과 문서 분할 방식에 맞춰 각각의 검색 인덱스를 구축했다. 공통 입력과 방식별 인덱스의 식별 정보는 부록의 근거표에 정리했다.

각 방식은 답변 생성과 GPT-5 nano 판정을 포함해 3회 평가했다. 최신 API 재현 기록은 150번의 질문 처리가 모두 정상 상태였다고 보고했다. 직접 실행 방식은 세 회차의 답변 50개가 완전히 같았으나 판정 점수는 달랐다. 평가셋과 판정기 조건을 맞춘 결과를 비교하되 자동 판정의 불확실성을 함께 해석한다.

## 8.2 검색·답변·출처 결과

표 10. 최종 평가 결과

| 측정 항목 | **API** 활용 방식 | 모델 직접 실행 방식 |
|---|---|---|
| 문서 Recall@5 | 1.000 | 1.000 |
| MRR@5 | 1.000 | 1.000 |
| 근거 단서 적중률 | 1.000 | 0.950 |
| 답변 점수, 3회 평균 | 0.996 | 0.849 |
| 내용 점수, 3회 평균 | 0.996 | 0.897 |
| 정답 문서 인용률 | 1.000 | 0.952 |
| 답 없는 질문 거절 | 8/8 | 7/8 |
| 답 있는 질문의 잘못된 거절 | 0 | 0 |

검색 지표는 답이 있는 42문항 기준이며, 근거 단서는 적용 가능한 질문에서만 평가한다. API 활용 방식의 근거 단서 값은 최초 최종 평가 기록에서, 답변 점수와 응답 시간은 이후 같은 구성의 재현 평가 기록에서 가져왔다. 답변 점수는 부분 정답과 출처 조건을 반영한 평균이다.[2: 114·145·147~149행]

정답 문서를 찾은 뒤에도 필요한 근거 조각의 확보와 답변 내용·출처를 함께 확인해야 했다. 직접 실행 방식의 답변 점수는 내용 점수보다 낮아 출처 규칙 미충족도 일부 반영됐다. 두 점수를 함께 살펴 내용과 출처 규칙이 평가에 미친 영향을 확인했다.

## 8.3 반복 평가와 자동 판정 변동

표 11. 최종 회차별 답변 점수

| 회차 | **API** 활용 방식 최신 재현 | 모델 직접 실행 방식 |
|---|---|---|
| 1회 | 1.000 | 0.881 |
| 2회 | 1.000 | 0.869 |
| 3회 | 0.988 | 0.798 |
| 평균 | 0.996 | 0.849 |

직접 실행 방식은 답변이 동일한데도 판정 점수가 0.798~0.881로 달랐다. 동일한 답변에 대한 자동 판정의 변동을 확인했으며, 설정 비교에서는 반복 점수의 범위와 문항별 답변을 함께 검토했다.

이전 100개 문서 평가 평균은 0.825, 최종 98개 문서 평가는 0.849였으며 두 값의 차이는 관찰된 판정 변동 범위 안에 있었다. 문서 중복 제거 후 답변이 달라진 항목도 있어 입력 조건과 판정 변동을 함께 봐야 한다. API 최초 최종 평가와 재현 평가는 같은 평균 점수를 보였고, 회차별 점수와 응답 시간에는 차이가 있었다.

## 8.4 응답 시간

표 12. 최종 응답 시간

| 항목 | **API** 활용 방식 | 모델 직접 실행 방식 |
|---|---|---|
| P50, 회차별 값 평균 | 8.50초 | 13.36초 |
| P95, 회차별 값 평균 | 15.06초 | 33.35초 |
| 평균 응답 시간 | 9.59초 | 15.55초 |

이번 실행에서는 API 활용 방식이 중간 응답과 느린 응답 구간 모두 짧았다. 시간에는 질문 임베딩부터 답변 생성까지를 포함하나 초기 모델 로딩과 채점은 제외한다. 이 결과는 해당 실험 환경의 개별 요청 처리 시간을 보여주며, 동시 요청 성능은 후속 운영 검증에서 확인한다.

직접 실행 방식의 이전 100개 문서 실행보다 최종 98개 문서 실행에서 P50이 길어졌으나, 기록만으로 원인을 확정하지 못했다. API 재현의 시간 차이도 외부 응답·서버 상태 등 여러

요인의 영향을 받을 수 있다. 비용 예시의 CPU 구성은 별도로 응답 시간을 측정할 계획이다.

## 8.5 대표 답변의 원문 대조

대표 사례는 고려대학교 차세대 포털·학사정보시스템 구축 사업의 업체당 발표시간 구성 질문이다. 새 API 실행 답변은 총 50분 이내, 발표 30분과 질의응답 20분을 제시했다. 원문 제안요청서 236쪽 '제안 설명회'와 대조해 시간 구성뿐 아니라 변경 가능 조건과 PM 직접 발표 요건도 확인했다.[1]

이 사례는 2026년 10월 20일 새로 실행한 질문 한 건의 결과와 원문을 비교한 것이다. 수치와 그 수치에 붙은 조건을 함께 확인하는 서비스의 사용 목적을 보여준다.

## 8.6 오류와 거절 분석

직접 실행 방식의 최종 98개 문서 평가에서는 실제 낙찰 금액을 묻는 질문에 사업 예산을 낙찰 금액으로 답한 사례가 남았다. 예산은 입찰 전 문서에 있는 값이지만 낙찰 금액과 다른 항목이다. 문서에 관련 숫자가 있다는 이유만으로 질문의 대상을 대체한 오류이며, 답이 없는 8문항 중 1문항 거절 실패로 집계됐다.

같은 질문의 과거 100개 문서 결과에는 '실제 낙찰 금액은 확인할 수 없다'는 답변을 거절 규칙이 인식하지 못한 사례가 있었다. 최종 98개 문서 평가에서는 실제로 잘못된 값을 단정해, 낙찰 금액과 예산의 항목 구분을 답변 생성 단계에서 보완할 과제로 확인했다.

탐색용 어려운 질문에서는 같은 문서의 본문과 일반조건이 서로 다른 기간을 제시하는 문제, 비교 질문에서 특정 사업의 근거가 빠지는 문제와 전체 집계 한계도 관찰됐다. 일부 평가 후보 자체에 정답 문서 누락 등 문제가 있어 그 점수를 최종 성능으로 사용하지 않았다. 실패 사례는 독립적으로 검수한 후 향후 평가에 반영할 필요가 있다.[2]

# 9. 독립 운영을 가정한 비용 분석

## 9.1 계산 목적과 가정

프로젝트는 제공받은 서버에서 두 방식을 개발·실행했다. 여기서는 지원 없이 각 방식을 독립적으로 운영할 때를 가정해 부분 비용을 계산한다. 서버와 답변 생성 비용을 대상으로 한 계산 예시다. 공통 가정은 미국 중부 지역, 온디맨드 서버, 월 720시간 가동과 질문 1,000회다.

서버 단가는 2026년 10월 20일 확인한 Google Cloud 공개 요금표를 사용한다. 직접 실행은 실제 사용한 L4 서버 사양을 유지하고, API 활용은 검색·재정렬을 GPU 없는 CPU 서버로 옮기는 가상 구성을 둔다. 생성 API 비용은 이전 모델 비교에서 산정한 질문당 비용을 적용한다.[1][11][12]

## 9.2 방식별 계산

표 13. 독립 운영 부분 비용 예시

| 항목 | API 활용 방식 | 모델 직접 실행 방식 |
|---|---|---|

| 서버 가정 | e2-standard-4, 4 vCPU·16GiB, GPU 없음 | g2-standard-4, 4 vCPU·16GiB, L4 1개 |
|---|---|---|
| 서버 시간당 단가 | 0.13402284달러 | 0.706832276달러 |
| 720시간 서버 비용 | 96.50달러 | 508.92달러 |
| 답변 생성 API / 질문 1,000회 | 약 2.50달러 | 외부 생성 API 호출 없음 |
| 합계 예시 | 약 98.99달러, 반올림 약 99달러 | 약 508.92달러, 반올림 약 509달러 |

직접 실행 서버 비용은 0.706832276 × 720 = 508.91923872달러다. API 활용 서버 비용은 0.13402284 × 720 = 96.4964448달러다. 답변 생성 예시는 이전 GPT-5 mini 356문항 계산 비용 0.889달러를 356으로 나눠 질문당 약 0.002497달러로 산정하고, 질문 1,000회에 약 2.497달러를 적용했다.

월 질문 수를 N이라고 하면 API 활용 부분 비용 예시는 96.4964448 + 0.00249719 × N달러로 표현한다. 직접 실행은 같은 서버·가동 시간을 유지한다는 조건에서 서버 부분 비용 508.91923872달러를 사용한다. 대규모 사용량에서는 처리량과 지연을 측정하고 필요한 서버 수를 반영해 비용을 다시 산정한다.

## 9.3 제외 비용과 해석 한계

계산에서 디스크, 네트워크, IP, 세금, 할인·크레딧, 문서와 질문의 임베딩, 평가 판정 API, 모니터링·운영 인건비를 제외했다. 입력·출력 길이, 캐시 적용, 실패·재시도와 단가 변경도 실제 비용에 영향을 준다. 직접 실행 방식은 생성에 필요한 연산 비용을 서버 비용으로 부담하며, 자동 평가를 수행하면 판정 API 비용이 추가된다.

API 활용 방식의 e2-standard-4는 비용 산정을 위한 가상 구성이다. 실제 도입 시 CPU 환경의 응답 시간·메모리·동시성을 검증하고, GPU가 필요하면 서버 비용을 다시 산정한다. 이 비용 예시는 저사용량 운영의 서버비 차이를 검토하는 출발점이다.

# 10. 성과, 한계 및 향후 과제

## 10.1 구현 성과

원본 100건의 본문과 표를 추출하고 중복을 제거한 공통 98개 문서에서 질문에 답과 출처를 제공하는 두 구현을 완성했다. 모델 API 연결뿐 아니라 모델 직접 실행, GPU 자원 관리, 방식별 실행 환경과 평가 흐름도 구성했다. 검색 문서 식별과 근거 조각 확보, 답변 내용과 출처, 거절을 분리해 평가했다.

공통 최종 평가에서 두 방식 모두 문서 Recall@5와 MRR@5가 1.0이었다. API 활용 방식은 답변 점수와 응답 시간에서 더 나은 결과를 보였으며, 직접 실행 방식에서도 출처를 제공하는 질의응답과 모델 운영 경험을 확보했다. 이번 결과를 바탕으로 실제 업무에서의 활용성과 검토 시간 절감을 확인하는 단계로 확장한다.

## 10.2 선택 과정에서 확인한 점

좋은 문서를 찾는 일과 답에 필요한 문단을 확보하는 일은 다르다. 후보 검색에서 빠진 조각은 재정렬만으로 복구하기 어렵고, 문서별 결과 제한은 다양성을 높이면서도 한 사업의

세부 근거를 제거할 수 있다. 이 때문에 문서 Recall 하나가 아니라 근거 전달과 실제 답변을 함께 확인해야 했다.

각 모듈의 단독 실험 결과도 통합 조건에서 재검토해야 했다. 의미 검색에서 이점이 있던 임베딩 혼합이 최종 검색에서 개선되지 않았고, 문서 분할·압축·간결화도 품질과 자원 측면의 이점이 항상 함께 나타나지 않았다. 최종 선택은 점수 하나의 최대화보다 필요한 근거·출처·안정성의 균형을 기준으로 했다.

## 10.3 평가의 한계

최종 평가는 50문항으로 한정되고 개발 과정에서 유사한 질문을 반복 사용했다. 설정 선택에 활용한 자료를 다시 평가한 결과에는 낙관적 편향이 있을 수 있다. 서로 다른 두 구현이 여러 구성 요소에서 함께 달라 어떤 변경이 성능 차이를 만들었는지 완전히 분리하지 못했다.

설명형 자동 판정은 같은 답변에도 다른 점수를 주었다. 정답 문서 인용률과 근거 단서 적중률은 각각 출처 연결과 단서 확보를 평가한다. 문장별 사실성과 조건의 완전성은 전문가 원문 검수와 독립 사용자 평가로 보완할 필요가 있다.

응답 시간은 해당 서버와 실행 조건에서 측정했다. 후속 검증은 동시 요청, 긴 질문과 근거, 서버 시작, API 장애와 운영 복구로 확장한다. 공통 모듈 정리 이후의 코드는 기존 최종 평가 조건과 대응시켜 확인한다.

## 10.4 향후 개선 과제

우선 새로운 문서와 독립 질문을 확보하고, 원문에서 정답·예외 조건·출처를 검수한 평가셋을 분리한다. 설명형은 여러 판정 결과와 사람 검토를 함께 활용해 경계 사례를 정리한다. 사업 예산과 낙찰 금액처럼 비슷한 항목의 혼동, 문서 내부의 상충 조건, 두 사업 비교와 전체 집계 질문을 별도 유형으로 다룬다.

검색 단계별로 후보 누락·재정렬 실패·문서별 제한 손실·생성 누락을 나눠 진단하고, 근거 범위가 부족한 경우에만 추가 검색하는 방법을 검토한다. 문서와 질문이 바뀌었을 때도 동일한 선택 규칙이 유지되는지 확인한다.

운영 측면에서는 CPU 재정렬의 지연·메모리·동시성을 측정하고, 예상 사용량을 반영한 비용을 다시 산정한다. 다회차 대화, 사용자 화면과 실제 업무 검토 시간을 검증해 시스템 품질을 사용자 관점의 성과와 연결한다.

## 10.5 결론

본 프로젝트는 제안요청서에서 답과 근거를 함께 찾는 RAG 시스템을 API 활용 방식과 모델 직접 실행 방식으로 구현했다. 각 방식에서 여러 모델과 설정을 비교해 최종 구성을 정하고 동일한 문서와 평가셋으로 품질·출처·거절·응답 시간을 확인했다. 현재 평가에서는 API 활용 방식이 더 높은 답변 점수와 짧은 응답 시간을 보였고, 모델 직접 실행 방식은 모델 운영과 자원 관리까지 포함한 구현 경험을 제공했다.

후속 구성 선택에는 독립 평가, 요구 응답 속도와 사용량, 실제 운영 환경의 검증 결과를 함께 반영한다. 원문 근거를 확인할 수 있는 구조를 유지하고, 확인할 수 없는 정보는 답하지 않는 품질을 지속적으로 개선한다.

# 참고자료

[1] 최종 발표 자료
[2] 실험 기록표, 실험 기록 탭
[3] 중간 발표 보고서와 EDA 기록
[4] 최종 통합 코드
[5] 초기 65문항 베이스라인 평가
[6] API 활용 방식 검색 설정 비교 보고서
[7] GPT-5 mini 비교 실행 기록
[8] 임베딩 모델 비교 보고서
[9] 최종 검색 조건의 청킹과 임베딩 혼합 비교
[10] 개요 추가 전체 재임베딩 비교
[11] Google Cloud 가속기 최적화 VM 요금
[12] Google Cloud 범용 VM 요금

성능 수치는 2026년 10월 20일의 공통 조건 3회 평가를 기준으로 한다. 부록의 실행 안내는 2026년 10월 21일 최신 dev 구조를 반영했다. 코드 버전 대응표는 최신 실행 코드와 측정 당시 코드를 구분하며, 과거 선택 실험은 당시 버전 그대로 보존한다.

# 부록 1. 실험 조건과 근거 대응표

코드 버전 대응표

| 변경·측정 내용 | 코드 버전 | 관련 실험·자료 |
|---|---|---|
| 최신 패키지 구조·실행 환경 명칭·설치 안내 | 527bfcc (2026-10-21) | pipelines/server·pipelines/api, .venv-server·.venv-api, README [4] |
| 이전 통합 단계의 공통 모듈 정리 | 2de6a5b | 당시 공통 모듈 분리·실행기 통합 |
| 초기 베이스라인 평가 | d767f3a | dyk-0005, 초기 65문항 [5] |
| API 활용 방식 최종 재현 평가 | 0fbd41d | yjk-0004~0006, 기록표 145행 |
| 모델 직접 실행 방식 공통 98개 문서 평가 | e8464ac | shn-0106~0108, 기록표 147~149행 |

표 14. 핵심 실험 대응

| 구분 | 실험·근거 | 조건과 사용 범위 |
|---|---|---|
| 초기 베이스라인 | dyk-0005, [5] | 추출 성공 98개·초기 65문항, 초기 기준선 |
| API 길이·중복 | dyk-0006~0010, 기록표 46~50행 | 98개·구 50문항 ded1e0b3, 답변은 700·1,000자만 |
| API 모델 비교 | dbp-0042·0043, 기록표 66~67행 | 356문항, 인용 350문항, 당시 lexical 재정렬 |
| 직접 실행 임베딩 | chy-0029·0030, [8] | 100개·65문항, dense 전용, 각 3회 |

| | | |
|---|---|---|
| 직접 실행 답변 모델 | 기록표 79~80행 | 구 50문항 ded1e0b3, 저장 검색 자료 비교 |
| 개요 추가 검색 | chy-0020, [10] | 100개·50문항과 별도 6문항, 근거 대상 20문항 |
| 개요 추가 답변 | pr37_generation_feedback.md | 100개·수정 50문항, 담당자 공유 비교 |
| API 최초 최종 평가 | shn-0102~0104, 기록표 114행 | 98개·수정 50문항, 각 회 새 판정 |
| API 최신 재현 | yjk-0004~0006, 기록표 145행 | 98개·수정 50문항, 최종 시간·점수 기준 |
| 직접 실행 최종 | shn-0106~0108, 기록표 147~149행 | 공통 98개·수정 50문항, 전체 3회 |

표 15. 최종 입력 식별값

| 대상 | 확인된 식별값 앞부분 | 비고 |
|---|---|---|
| 공통 문서 | f7a2740b | 중복 제거 후 98개 |
| 최종 평가셋 | 67506c51 | eval_v4_20261015, 50문항 |
| API 활용 방식 인덱스 | 7d0e8bac | 11,551개 조각 |
| 모델 직접 실행 방식 인덱스 | 95aa8191 | 10,008개 조각 |
| 비교 실험의 과거 문서 | 97bb7a5a | 100개, 최종 98개와 구분 |
| 비교 실험의 구 50문항 | ded1e0b3 | 수정 최종 평가셋과 구분 |

실험 번호는 실행 경로·담당자와 함께 식별한다. 방식별로 번호를 새로 부여한 기록은 이 세 정보를 묶어 대응시킨다. 파일 해시 앞부분은 대응을 돕기 위한 표기이며, 재현 검증에서는 전체 해시를 대조한다.

# 부록 2. 실행 및 재현 절차

최신 dev 기준은 2026년 10월 21일의 실행 구조다. 프로젝트 루트에서 실행하며, 모델 직접 실행 방식과 API 활용 방식은 각각의 가상환경과 설정 파일을 사용한다. 아래 절차는 최신 코드의 설치·실행 안내이며, 최종 평가 당시의 수치와 연결되는 코드·입력은 부록 1에서 확인한다.

## 1. 실행 환경과 입력 자료 준비

Python 3.12.3과 GCP Linux·NVIDIA L4를 팀 실행 기준으로 사용한다. 원본은 data/raw/data_list.csv와 data/raw/files/에 준비한다. 최종 평가 질문 파일은 팀 공유 자료에서 받아 아래 예시의 data/eval_v4.json 위치에 둔다. 파일명과 관계없이 표 15의 최종 평가셋 식별값(67506c51)과 일치하는 수정 50문항을 사용한다.

```
python3.12 -m venv .venv-server

.venv-server/bin/python -m pip install -r requirements-server.txt

.venv-server/bin/python -m pip check
```

```
python3.12 -m venv .venv-api
.venv-api/bin/python -m pip install -r requirements-api.txt
.venv-api/bin/python -m pip check
```

## 2. 모델과 설정 파일 준비

모델 직접 실행 방식은 models/generate/generation_model, models/embedding/default_model, models/rerank/bge-reranker-v2-m3에 생성·임베딩·재정렬 모델을 준비한다. 다운로드 명령과 고정 모델 버전은 저장소 README의 모델 준비 절차를 따른다.[4] 설정 예시 파일을 복사한 뒤 실제 API 키와 담당자 정보를 입력한다.

```
test -f .env.server || cp .env.server.example .env.server
test -f .env.api || cp .env.api.example .env.api
.venv-server/bin/python -m pipelines.server.server_run check-config
.venv-api/bin/python -m pipelines.api.api_run check-config
```

모델 직접 실행 방식은 .env.server의 SERVER_MODELS_DIR=models, SERVER_GENERATION_MODEL=generation_model, SERVER_EMBEDDING_MODEL=default_model, SERVER_RERANK_MODEL=bge-reranker-v2-m3를 사용한다. API 활용 방식은 .env.api의 OPENAI_API_KEY와 임베딩·생성 모델을 설정한다. 자동 판정을 수행할 때에는 모델 직접 실행 방식에도 판정용 OPENAI_API_KEY가 필요하다. check-config는 설정을 읽는 점검 명령이다.

## 3. 공통 문서와 방식별 인덱스 구축

공통 파싱은 한 번 수행하고 두 방식의 인덱스를 각각 구축한다. 최종 평가를 재현할 때에는 보관된 공통 98개 문서와 짝이 맞는 인덱스를 사용한다. 새 문서로 구축하는 경우 아래 명령을 실행하고 문서 수와 처리 내역을 확인한다. 기본 인덱스는 indexes/server/와 indexes/api/로 분리된다.

```
.venv-api/bin/python -m pipelines.api.api_run parse
.venv-server/bin/python -m pipelines.server.server_run build --processed-dir
data/processed/common
.venv-api/bin/python -m pipelines.api.api_run build --processed-dir data/processed/common
```

공통 문서는 data/processed/common/documents.json에 저장하며, 추출 실패와 중복 제외 내역은 같은 폴더의 parsing_errors.json에서 확인한다. 기존 문서·인덱스는 실험별로 보존하고 새 구축에는 별도 경로를 지정할 수 있다. --limit을 생략하면 전체 원본을 처리한다.

## 4. 공통 조건 평가와 결과 확인

같은 평가 질문 파일을 두 방식에 지정하고 각각 3회 실행한다. 최종 평가와 같은 반복 채점 조건은 GPT-5 nano 판정기를 사용하며 판정 캐시 옵션을 생략한다. 입력 파일과 문서·인덱스의 식별값, 모델·청킹 설정과 실행 코드를 결과와 함께 기록한다.

```
.venv-server/bin/python -m pipelines.server.server_run evaluate --processed-dir
data/processed/common --eval-file data/eval_v4.json --judge-model gpt-5-nano
```

```
.venv-api/bin/python -m pipelines.api.api_run evaluate --processed-dir
data/processed/common --eval-file data/eval_v4.json --judge-model gpt-5-nano
```

결과 JSON은 results/server/와 results/api/에, 요약 보고서는 각 폴더의 reports/에 저장한다. 실행 번호 상태도 .experiment-state/server/와 .experiment-state/api/에서 각각 관리한다.

두 방식을 연속 비교하려면 python3.12 run.py evaluate --eval-file data/eval_v4.json --judge-model gpt-5-nano를 실행하고 메뉴에서 3 또는 server+api를 선택한다. 서버 모델 평가 후 API 평가를 수행하고 두 결과의 비교표를 출력한다. 같은 옵션이 양쪽에 전달되므로 인덱스와 결과 저장 경로는 방식별 기본 분리 경로를 사용한다.

## 5. 최종 평가 당시 결과의 재현

최종 성능 수치의 재현은 부록 1의 측정 당시 코드와 보관 입력을 기준으로 한다. API 활용 방식은 공통 문서·평가셋·인덱스에 대응하는 재현 3회 기록을, 모델 직접 실행 방식은 동일 98개 문서로 다시 구축한 인덱스의 3회 기록을 사용했다. 원시 결과와 전체 해시는 해당 실험 기록의 보관 경로에서 확인한다.

최신 패키지 구조로 재실행한 결과는 새 실험으로 기록해 과거 측정과 비교한다. API 활용 방식의 인덱스 구축·질문 처리와 두 방식의 자동 판정에는 API 사용량이 발생한다.